Marwadi University

Vidhya Popat

Programming Practices -2 [BCA - 2]

UNIT 3 – POINTER

POINTER IN C

In C programming, a pointer is a variable that stores the memory address of another variable. Unlike most other variables that directly contain a value, a pointer contains the address of a value. Think of it as a way to indirectly refer to a location in memory where a specific value is stored.

DECLARE POINTER IN C

To declare a pointer variable, you use an asterisk (*) before the variable name.

The general declaration of a pointer is:

data-type *pointer-name;

int *ptr;

This declares a pointer named ptr that can point to an integer.

ASSIGNING POINTER

You can assign the address of a variable to a pointer using the address-of operator '&'.

For example:

```
int num = 10;
int  *ptr;
ptr = &num;
```

Now, ptr contains the memory address where the integer variable num is stored.

ACCESSING POINTER

%p format specifier is used to display address of any variable in printf function.

For example :

```
printf("Address of  num variable is : %p ",&num);
```

ACCESSING POINTER

Indirection Operator and Dereferencing Operator (*)

Imagine you have a box called num, and inside that box, there's a number written on a piece of paper. Now, a pointer is like a signpost that tells you where this box num is located.

So, in simpler terms, the * symbol helps you access the value stored in the box that the pointer is pointing to. It's like saying, "Hey, pointer, show me what's inside the box you're pointing to!"

For example : In the following program, the value of the integer variable num is changed twice.

EXAMPLE

```
#include<stdio.h>
#include<conio.h>
void main()
{
        int num = 5;
        int *iptr  = &num;
        printf("\n Value of num is : %d",num);
        num = 10;
        printf("\n Value of num is : %d",*iptr);
        *iptr  = 15;
        printf("\n Value of num is : %d",num);
}
```

Out put :

Value of num is : 5
Value of num is : 10
Value of num is : 15

POINTER IN C

One thing to remember always is that the data type of the pointer variable and the variable whose address it will store must both be of the same type. Therefore, the following code is not valid.

```
int x = 10;
float y = 2.0;
int *px;
float *py;
px = &y;//INVALID
py = &x;//INVALID
```

POINTER IN C

Also note that it is not necessary that the pointer variable will point to the same variable throughout the program. It can point to any variable as long as the data type of the pointer variable is same as that of the variable it points to. The following code illustrates this concept.

```
#include <stdio.h>
void main()
{
    int a=3, b=5;
    int *pnum;
    pnum = &a;
    printf("a = %d", *pnum);
    pnum = &b;
    printf("b = %d", *pnum);
}
```

Output

a = 3 b = 5

POINTER EXPRESSIONS AND POINTER ARITHMETIC

Like other variables, pointer variables can also be used in expressions. For example, if ptrl and ptr2 are pointers, then the following statements are valid.

```
int numl=2, num2= 3, sum=0
int *ptrl, *ptr2;
ptrl = &numl;
ptr2 = &num2;
sum = *ptrl + *ptr2;
printf(“sum = %d", sum);
```

POINTER EXPRESSIONS AND POINTER ARITHMETIC

When using pointers, unary increment (++) and decrement (--) operators have greater precedence than the dereference operator (*). But both these operators have a special behaviour when used as a suffix.

*ptr++ is equivalent to *(ptr++) as ++ has greater operator precedence than *. Therefore, the expression will increase the value of ptr so that it now points to the next element. This means the statement *ptr++ does not perform the intended task. Therefore, to increment the value of the variable whose address is stored in ptr, **you should write (*ptr)++**

RULES FOR POINTER OPERATIONS

- A pointer variable can be assigned the address of another variable **of the same type.**
- A pointer variable can be initialized with **a NULL (or 0) value.**
- Prefix or postfix increment and decrement operators can be **applied to a pointer variable. (*ptr)++**
- An integer value can be **added or subtracted from a pointer variable.**
- **A pointer variable can be compared with another pointer variable of the same type using relational operators.**

NULL POINTERS

We have seen that the pointer variable is a pointer to some other variable of the same data type. However, in some cases we may prefer to have null pointer which is a special pointer value that does not point anywhere.

This means that a null pointer does not point to any valid memory address.

There is difference between NULL pointer and uninitialized pointer.
NULL pointer points to nothing and uninitialized pointer points to any garbage address.

NULL POINTERS

To declare a null pointer you may use the predefined constant NULL, which is defined in several standard header files including <stdio.h>, <stdlib.h>, and <string. h>. After including any of these files in your program just write,

```
int *ptr = NULL;
```

You may also initialize a pointer as a null pointer by using a constant 0(zero), as shown below.

```
int *ptr = 0;
```

GENERIC POINTERS

A generic pointer is a pointer variable that has void as its data type.

The void pointer, or the generic pointer, is a special type of pointer that can be used to point to variables of any data type.

It is declared like a normal pointer variable but using the void keyword as the pointer's data type.

For example,
void *ptr;

GENERIC POINTERS

You need to type cast void pointer(generic pointer) to another kind of pointer before using it.

```
#include <stdio.h>
void main()
{
    int x=10;
    char ch = 'A';
    void *gp;
    gp = &x ;
    printf("Generic pointer points to the integer value = %d", *(int*)gp);
    gp = &ch;
    printf("Generic pointer now points to the character %c”, *(char*)gp);
    }
```

Output

Generic pointer points to the integer value = 10

Generic pointer now points to the character = A

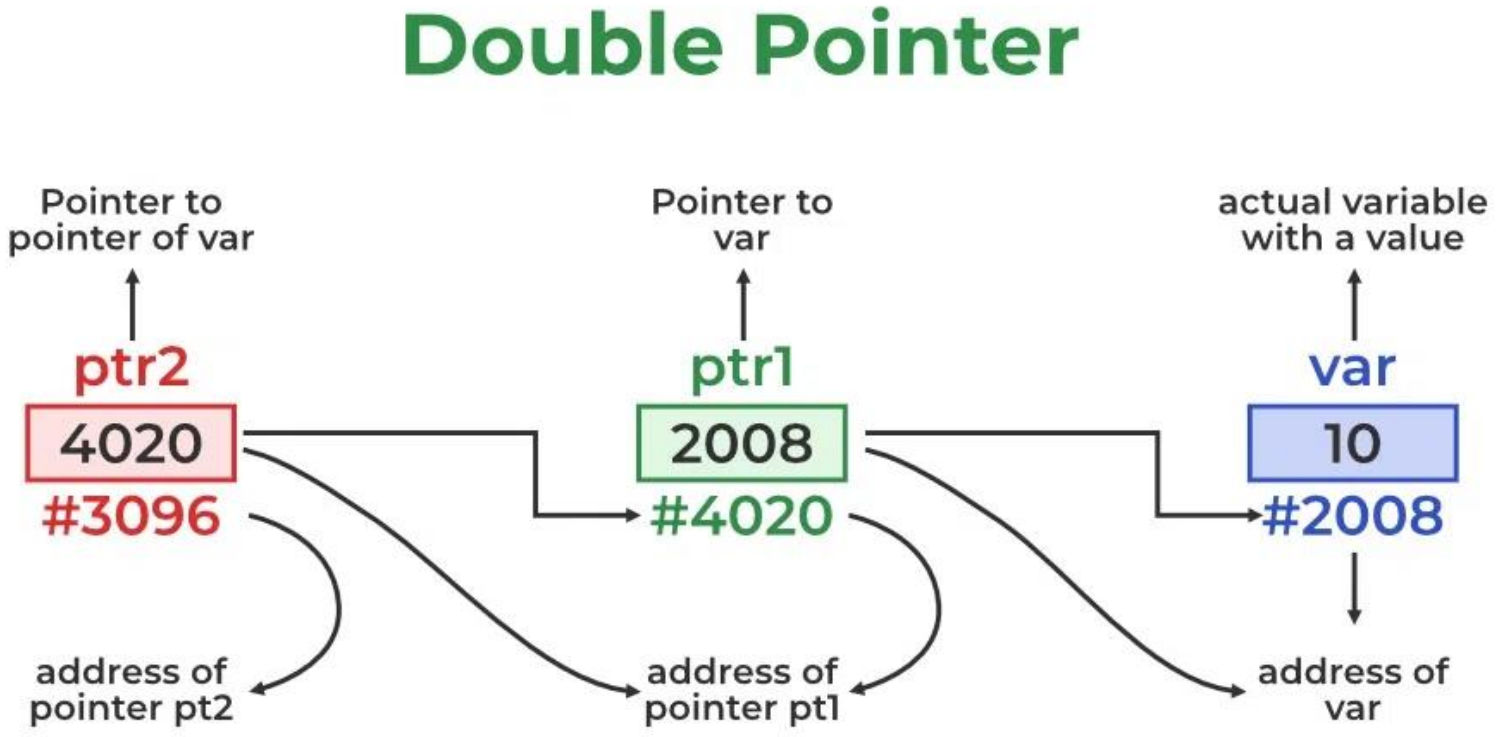
POINTER TO POINTER (DOUBLE POINTER/POINTER CHAINING)

A pointer to a pointer is a form of multiple indirection, or a chain of pointers.

Normally, a pointer contains the address of a variable.

When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below.

POINTER TO POINTER (DOUBLE POINTER/POINTER CHAINING)



POINTER TO POINTER (DOUBLE POINTER)

A variable that is a pointer to a pointer must be declared as such.
This is done by placing an additional asterisk in front of its name.
For example, the following is the declaration to declare a pointer to a pointer of type int:

```
int **var;
```

When a target value is indirectly pointed to by a pointer to a pointer, accessing that value requires that the asterisk operator be applied twice, as is shown below in the example:

POINTER TO POINTER (DOUBLE POINTER)

```
#include<stdio.h>

void main()
{
	int i =50;
	int **ptr1;
	int *ptr2;
	clrscr();
	ptr2 = &i;
	ptr1 = &ptr2;
	printf(“The value of **ptr1 : %d",**ptr1);
	printf("The value of *ptr2 : %d",*ptr2);
}
```

Output :

The value of **ptr1 : 50

The value of *ptr2 : 50

POINTER INCREMENTS AND SCALE FACTOR

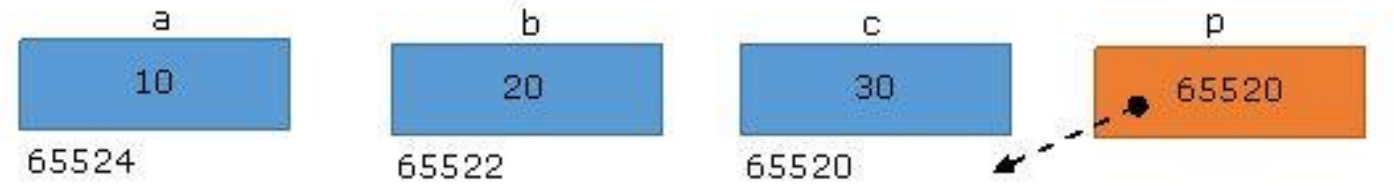
In case of primitive types adding 1 to the variable results incrementing of its value by 1 and subtracting 1 to the variable results decrementing of its value by 1.

Since pointer is not a normal variable adding 1 to the pointer results next variable's address. Subtracting 1 to the pointer results previous variables address.

Now we will observe the same with an example. Say for example a series of three variables of short type are declared and a pointer is stored with the address of a variable

POINTER INCREMENTS AND SCALE FACTOR

Here p is the pointer to the variable c, holds the address of c 65520. Now p+1 gives the address of next variable b 65522 and p+2 gives the address of next variable a 65524.



POINTER INCREMENTS AND SCALE FACTOR

```
#include<stdio.h>
void main()
{
int a=10,b=20,c=30;
int *p=&c;
printf("p     %u",p);
printf("p+1 %u",p+1);
printf("p+2 %u",p+2);
getch();
}
```

Output:

```
p        65520
p+1      65522
p+2      65524
```

POINTER INCREMENTS AND SCALE FACTOR

If we take a float pointer then increments by 4 on adding 1 to the pointer.

```
#include<stdio.h>
void main()
{
	float a=10.34,b=20.56,c=30.44;
	float *p=&c;
	printf("p %u",p);
	printf("p+1 %u",p+1);
	printf("p+2 %u",p+2);
	getch();
}
```

Output:

```
p	65514
p+1	65518
p+2	65522
```

POINTER INCREMENTS AND SCALE FACTOR

Scale factor

If we conclude the above observation, int pointer increments by 2 on adding 1, float pointer increments by 4 on adding 1 and char pointer increments by 1 on adding 1 that is pointer is incrementing by its type size on adding 1

Scale factor is the quantity of increment in the value of a pointer on adding 1

PASSING ARGUMENTS TO FUNCTION USING POINTERS

C programming language allows you to pass a pointer to a function.

To do so, simply declare the function parameter as a pointer TYPE.

One of the best things about pointers is that they allow functions to alter variables outside of their scope.

By passing a pointer to a function you can allow that function to read and write to the data stored in that variable.

PASSING ARGUMENTS TO FUNCTION USING POINTERS

Two ways to pass arguments to function :

Call by value (passing a normal variable as an argument)
Call by reference (passing pointer variable as an argument)

CALL BY VALUE

Call by value :

The call-by-value method of passing arguments to a function copies the actual value of an argument into the formal parameter of the function.

In this case, changes made to the parameter inside the function do not affect the argument.

By default, C programming language uses call by value method to pass arguments. In general, this means that code within a function cannot alter the arguments used to call the function.

CALL BY VALUE

```
#indude <stdio.h>

void swap(int x, int y); // function declaration

void main ()
{
int a =100; int b = 200;  // local variable definition

printf("Before swap, value of a : %d", a );
printf("Before swap, value of b : %d", b );

swap(a, b); // calling a function to swap the values

printf("After swap, value of a : %d", a );
printf("After swap, value of b : %d", b );
}
```

CALL BY VALUE

```
void swap(int x, int y) // function definition to swap the values
{

int temp;
temp = x;
x= y;
y = temp;

}
```

Output :

Before swap, value of a :100 Before swap, value of b :200

After swap, value of a :100 After swap, value of b :200

CALL BY REFERENCE

Call by Reference :

The call-by-reference method of passing arguments to a function copies the address of an argument into the formal parameter.

Inside the function, the address is used to access the actual argument used in the call.

This means that changes made to the parameter affect the passed argument.

To pass the value by reference, argument pointers are passed to the functions just like any other value.

CALL BY REFERENCE

```
#indude <stdio.h>

void swap(int *x, int *y); // function declaration

 void main ()
{
      int a =100; int b = 200; // local variable definition

      printf("Before swap, value of a : %d", a );
      printf("Before swap, value of b : %d", b );

      swap(&a, &b); /* calling a function to swap the values. * &a indicates pointer to a ie.
      address of variable a and * &b indicates pointer to b ie. address of variable b. */

      printf("After swap, value of a : %d", a );
      printf("After swap, value of b : %d", b );
}
```

CALL BY REFERENCE

```
/* function definition to swap the values */
void swap(int *x, int *y)
{
      int temp; temp = *x;
      *x = *y;
      *y = temp;

}
```

Output :

Before swap, value of a :100 Before swap, value of b :200
After swap, value of a :200 After swap, value of b :100

DIFFERENCE

Key Takeaways

Method	Effect	Changes Original Variable?
Call by Value (`swap(int x, int y)`)	Copies are modified	❌ No
Call by Reference (`swap(int *x, int *y)`)	Original values are modified via pointers	✅ Yes

POINTER TO ARRAY

- The concept of array is very much bound to the one of pointer.
- **A pointer to an array in C is a pointer that holds the address of an array.**

This can be tricky because there's a difference between:

1. A pointer to the first element of an array.
2. A pointer to the entire array.

Both pointers access the same data but in different ways.

POINTER TO ARRAY

1. Pointer to the First Element of an Array:

When you create an array, its name acts as a pointer to the first element.

EXAMPLE:

```
int arr[5] = {10, 20, 30, 40, 50};

int *ptr = arr;            // ptr points to arr[0]
```

ptr stores the address of arr[0], meaning *ptr is 10.

ptr + 1 moves to the next element (arr[1]).

POINTER TO ARRAY

2. Pointer to the Entire Array:

To store the address of the whole array, you need to use a pointer to an array.

```
int arr[5] = {10, 20, 30, 40, 50};
int (*ptr)[5] = &arr;                    // Pointer to the entire array
```

ptr stores the address of arr, not just arr[0].
***ptr gives the entire array.**
(*ptr)[0] is 10.

POINTER TO ARRAY

Key Difference between both:

int *ptr = arr; → Pointer to the first element.
int (*ptr)[5] = &arr; → Pointer to the entire array.

POINTER TO ARRAY

```
#include <stdio.h>

void main() {
    int arr[5] = {10, 20, 30, 40, 50};

    int *ptr1 = arr;    // Pointer to the first element
    int (*ptr2)[5] = &arr; // Pointer to the entire array

    // Accessing elements using ptr1 (Pointer to first element)
    printf("Using ptr1 (Pointer to first element):\n");
    printf("First element: %d", *ptr1);
    printf("Second element: %d", *(ptr1 + 1));

    // Accessing elements using ptr2 (Pointer to entire array)
    printf("\nUsing ptr2 (Pointer to entire array):\n");
    printf("First element: %d", (*ptr2)[0]);
    printf("Second element: %d", (*ptr2)[1]);
}
```

POINTER TO ARRAY

OUTPUT:

Using ptr1 (Pointer to first element):
First element: 10
Second element: 20

Using ptr2 (Pointer to entire array):
First element: 10
Second element: 20

Both pointers access the same data but in different ways.

PASSING AN ARRAY TO A FUNCTION

When passing an array to a function, you can do it in two main ways:

1. Passing a pointer to the first element
2. Passing a pointer to the entire array

PASSING AN ARRAY TO A FUNCTION

Example Demonstrating Both Methods:

```
#include <stdio.h>

// Function 1: Receiving a pointer to the first element
void printElements(int *arr, int size)
{
    printf("Using pointer to first element:");
                for (int i = 0; i < size; i++)
               {
                printf("%d ", arr[i]);     // Same as *(arr + i)
               }
    printf("\n");
}
```

PASSING AN ARRAY TO A FUNCTION

```
// Function 2: Receiving a pointer to the entire array
void printWholeArray(int (*arr)[5])
{
    printf("Using pointer to entire array:\n");
            for (int i = 0; i < 5; i++)
            {
            printf("%d ", (*arr)[i]);                // Dereferencing the array pointer
            }
    printf("\n");
}
```

PASSING AN ARRAY TO A FUNCTION

```
int main()
{
    int arr[5] = {10, 20, 30, 40, 50};

    // Passing pointer to first element
    printElements(arr, 5);

    // Passing pointer to entire array
    printWholeArray(&arr);

    return 0;
}
```

PASSING AN ARRAY TO A FUNCTION

Explanation:

Passing a Pointer to First Element (printElements)

The function receives int *arr, which means arr is a pointer to the first element.
We access elements using arr[i] or *(arr + i).

Passing a Pointer to the Entire Array (printWholeArray)

The function receives int (*arr)[5], a pointer to an array of 5 integers.
We use (*arr)[i] to access elements.

PASSING AN ARRAY TO A FUNCTION

OUTPUT:

Using pointer to first element:
10 20 30 40 50

Using pointer to entire array:
10 20 30 40 50

Both approaches work, but the pointer to the first element is more commonly used because it allows flexible array sizes.

DIFFERENCE BETWEEN ARRAY NAME AND POINTER

Array	Pointer
1. Array : When memory is allocated for array, its base address is fixed and it cannot be changed during program execution. In other words, an array name is an address constant. Therefore, its value cannot be changed. To ensure that the address of the array does not get changed even accidentally, C does not allow array names to be used as an lvalue. Hence, array name cannot appear on the left side of the assignment operator.	1. **Pointer :** However, you may declare a pointer variable of appropriate type that points to the first element of the array and use it as lvalue.

DIFFERENCE BETWEEN ARRAY NAME AND POINTER

Array	Pointer

e.g. of array
 	
void main()
{
int arr[5],i;
for(i=0;i<5;i++)
{
*arr=0;
arr++; //error
}
}
	e.g. of pointer
	
	void main()
	{
	int arr[5],i,*parr;
	parr=arr;
	for(i=0;i<5;i++)
	{
	*parr=0;
	parr++;
	}
	}

DIFFERENCE BETWEEN ARRAY NAME AND POINTER

Array	Pointer

2. Array :
An array cannot be assigned to another array. This is because an array name
cannot be used as the lvalue.
 	
int arr[] = {1,2,3,4,5};
int arr2[5];
arr2=arr1; //ERROR
	2. But, one pointer variable can be assigned to another pointer variable of the same type. Therefore, the following statements are valid in C.
	
	int arr[]={1,2,3,4,5};
	int *ptr1, *ptr2;
	ptr1=arr1;
	ptr2=ptr1;
	Note : when we write ptr2=ptr1, we are not copying the data pointed to, we are just making two pointers point to the same location.

DIFFERENCE BETWEEN ARRAY NAME AND POINTER

Array	Pointer
3. When an address operator is applied to an array name, it gives the same value as the array reference without the operator. Therefore, arr and &arr gives the same value.	3. This is not true for pointer.
4. The sizeof operator when applied to an array name returns the no. of bytes allocated for the array.	4. The sizeof operator returns the no. of bytes used for the pointer variable.

ARRAY OF POINTERS

An array of pointers can be declared as

```
int *ptr[10];
```

The above statement declares an array of 10 pointers, where each of the pointer points to an integer variable.

For e.g., look at the code below.

```
int *ptr[10];
int p = 1,q=2,r=3,s=4,t=5;
ptr[0]=&p;
ptr[1]=&q;
ptr[2]=&r;
ptr[3]=&s;
ptr[4]=&t;
```

ARRAY OF POINTERS

Can you tell what will be the output of the following statement?

```
printf("\n %d", *ptr[3]);
```

ARRAY OF POINTERS

Yes, the output will be 4 because ptr[3] stores the address of integer variable s and *ptr[3] will therefore print the value of s that is 4.

ARRAY OF POINTERS

```
#include <stdio.h>
#include <conio.h>


void main ()
{
        int  var[] = {100,200,300};
        int i, *ptr[3];
        clrscr();
        for ( i = 0; i < 3; i++)
        {
                ptr[i] = &var[i]; /* assign the address of integer. */
        }

        for ( i = 0; i < 3; i++)
        {
                printf("Value of var[%d] = %d", i, *ptr[i] );
        }
        getch();
}
```

Output :

Value of var[0]=100

Value of var[1]=200

Value of var[2]=300

ARRAY OF POINTERS

Now consider an array of character pointers that is pointed to the strings.

```
char *ptr[3];
```

In the ptr array, each element is a character pointer.

Therefore, we can assign character pointers to the elements of the array.

For e.g. :

```
ptr[0]=str;
```

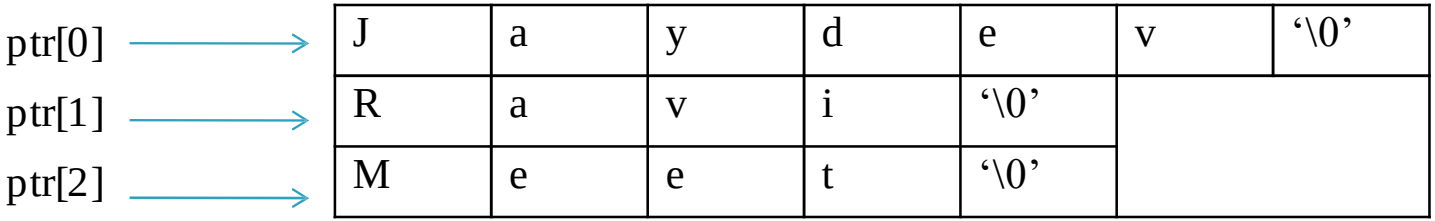
ARRAY OF POINTERS

Another way to initialize an array of characters with three strings can be given as,

char *ptr={“Jaydev”, “Ravi”, “Meet”};

Here, ptr[0] is Jaydev, ptr[1] is Ravi and ptr[2] is Meet.

ARRAY OF POINTERS



ARRAY OF POINTERS

Array of pointer	Pointer to an array
If we declare an array of pointer using, data_type *array_name[size];	If we declare a pointer to an array using, data_type (*array_name) [size];
Here, size represents the no. of rows and the space for columns that can be dynamically allocated.	Here, size represents the no. of columns and the space for rows that may be dynamically allocated.
Each element of the array is an address to memory location which holds the data.	It is base address of the array.

POINTERS AND 2-D ARRAYS

Elements of a two – dimensional array are stored in contiguous memory locations.

A two – dimensional array is not the same as an array of pointers to one - dimensional arrays.

To declare a pointer to two – dimensional array, you may write

```
int **ptr;
```

POINTERS AND 2-D ARRAYS

Individual elements of the array mat can be accessed using either:

mat[i][j] or *(*(mat + i) + j) or*(mat[i]+j);

See pointer to a one dimensional array can be declared as, int arr[]={1,2,3,4,5};

int *parr; parr=arr;

Similarly, pointer to a two dimensional array can be declared as, int
arr[2][2]={{1,2},{3,4}};
int (*parr)[2];
parr=arr;

POINTERS AND 2-D ARRAYS

/*Look at the code given below which illustrates the use of a pointer to a two dimensional array.*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
	int arr[2][2]={{1,2},{3,4}};
	int i, (*parr)[2],j;
	clrscr();
	parr=arr;
	for(i=0;i<2;i++)
	{
		for(j=0;j<2;j++)
		{
			printf(" %d", *(parr[i]+j));
		}
	}
	getch();
}
```

OUTPUT 1 2 3 4

POINTERS AND STRINGS

In C, strings are treated as arrays of characters that terminated with a binary zero character (written as '\0').

Consider, for e.g.,

```
Char str[10];  str[0]='M';
str[1]='E';
str[2]='F';
str[3]='\0'
```

POINTERS AND STRINGS

C language provides two alternative ways of declaring and initializing a string.

First,

```
char str[10] = {'M','E','F','\0'};
```

Second,

```
char str[10] = "MEF";
```

When double quotes are used, null character is automatically appended to the end of the string.

POINTERS AND STRINGS

```
#include <stdio.h>
#include <conio.h>
void main()
{
	char str [] = "Hello";
	char *pstr;
	pstr = str;
	printf("\n The string is: ");
	while(*pstr ! = '\0')
	{
		printf("%c”, *pstr);
		pstr++;
	}
	getch();
}
```

Output

The string is: Hello

FUNCTION POINTERS (POINTER TO FUNCTION)

Function pointers are pointer variables that point to the address of a function.

Like other pointer variables, function pointers can be declared, assigned values, and used to access the functions they point to.

This is a useful technique for passing a function as an argument to another function. In order to declare a pointer to a function we have to declare it like the prototype of the function except that the name of the function is enclosed between parentheses () and an asterisk (*) is inserted before the name.

FUNCTION POINTERS (POINTER TO FUNCTION)

The syntax of declaring a function pointer can be given as,
return_type (* function_pointer_name) (argument_li st);

For example, if fp is a function pointer and we have a function add () with prototype given as,

int add (int, int) ;

Then writing fp = add; initializes the function pointer fp with the address of add() .

FUNCTION POINTERS

```
#include<stdio.h>
#include<conio.h>

float (*func) (float,float); // Define function pointer
float add(float,float);
float sub(float,float);
void main()
{
	clrscr();
	func=add;
	//function pointer points to add
	printf("\n Addition = %f",func(9.5,3.1));
	func=sub;
	printf("\n Subtraction = %f",func(9.5,3.1));
	getch();
}
```

FUNCTION POINTERS

```
float add(float x,float y)
{
	return (x+y);
}
float sub(float x,float y)
{
	return (x-y);
}
/*output
Addition = 12.800000
Subtraction = 8.400000
*/
```

POINTER AND STRUCTURE

You can create pointer of structure variable and also you can point it. You need to use -> symbol instead of . Symbol for accessing member of structure.

```
#include <stdio.h>
struct person
{
   int age;
   float weight;
};
void main()
{
    struct person *personPtr, person1;
    personPtr = &person1;
    clrscr();
```

POINTER AND STRUCTURE

```
printf("Enter age: ");
    scanf("%d", &personPtr->age);
    printf("Enter weight: ");
    scanf("%f", &personPtr->weight);
    printf("Displaying:\n");
    printf("Age: %d\n", personPtr->age);
    printf("weight: %f", personPtr->weight);
    getch();
}
```

MEMORY ALLOCATION IN C PROGRAMS

C supports 3 kinds of memory allocation through the variables in C programs :

- Static allocation
- Automatic allocation
- Dynamic allocation

STATIC ALLOCATION

- When we declare a static or global variable, static allocation is done for the variable.
- Each static or global variable is allocated a fixed size of memory space.
- The no. of bytes reserved for the variable cannot change during execution of the program.

STATIC ALLOCATION

- Imagine you book a hotel room before your trip.
- You decide the number of rooms (memory size) in advance.

Example:

```
int numbers[5];       // Reserves 5 rooms (integers) in memory
```

Once booked, you can't increase or decrease the rooms.

AUTOMATIC ALLOCATION [STACK]

- When we declare an automatic variable, such as a function argument or a local variable, automatic memory allocation is done.

EXAMPLE:

Think of it as a locker in a gym:

- You get a fixed locker (memory) when you enter.
- You don’t need to manually give it back; it’s automatically released when you leave.
- Fast and efficient but limited in size.

AUTOMATIC ALLOCATION

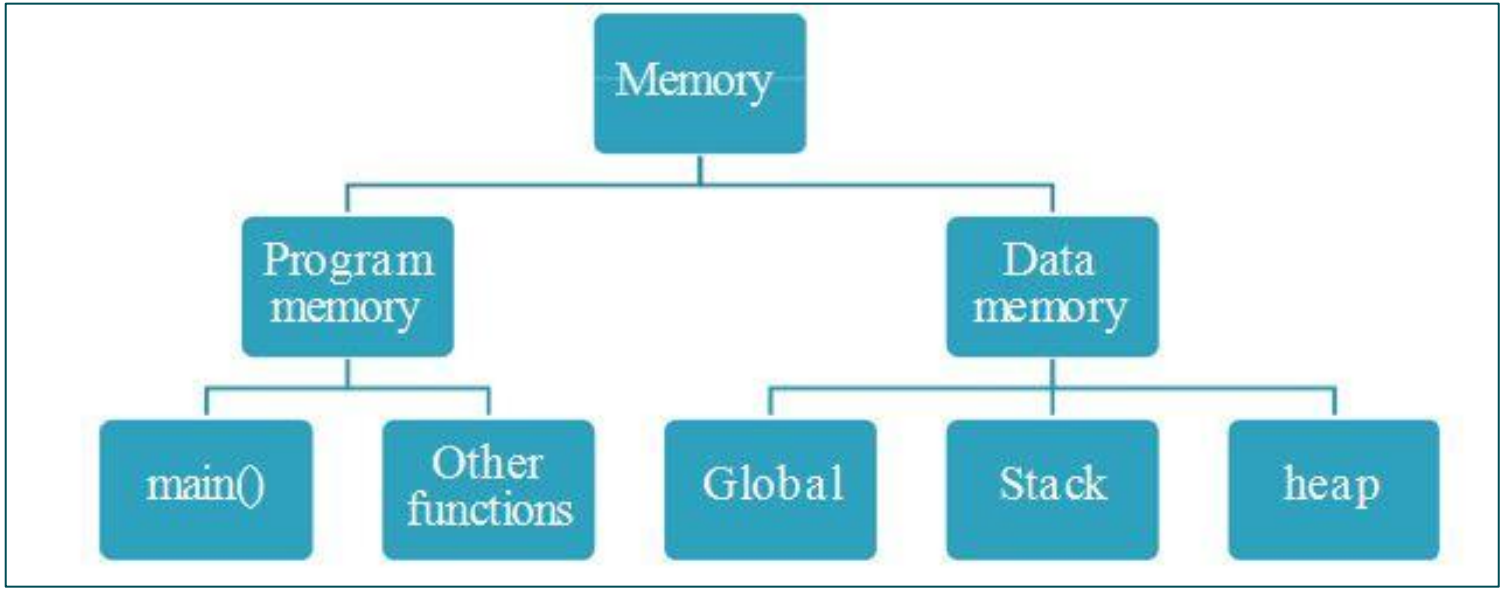
```
void example()

{
    int num = 10;                    // Stored in stack memory
    char name[ ] = "John";           // Stored in stack
}

 // Memory for x and name is freed when the function ends
```

DYNAMIC ALLOCATION

- This process of allocating memory to the variables during execution of the program or at run time is known as dynamic memory allocation.

MEMORY USAGE

- There are two parts of memory : program memory and data memory.



MEMORY USAGE

- The program memory consists of memory used for the main() and other called functions in the program, whereas data memory consists of memory needed for permanent definitions such as global data, local data, constants, and dynamic memory data.

- The way in which C handles the memory requirements is a function of the operating system and the compiler.

DYNAMIC MEMORY ALLOCATION

- The size of array you have declared initially can be sometimes insufficient and sometimes more than required.
- Dynamic memory allocation allows a program to obtain more memory space, while running or to release space when no space is required.
- There are 4 library functions under "stdlib.h" for dynamic memory allocation.

DYNAMIC MEMORY ALLOCATION

Dynamic Allocation (Flexible Rooms)

Here, you book rooms when needed and release them when done.

- C provides functions from <stdlib.h> for this:

1. malloc()
2. calloc()
3. realloc()
4. free()

DYNAMIC MEMORY ALLOCATION

Function	Use of Function
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	dellocate the previously allocated space
realloc()	Change the size of previously allocated space

DYNAMIC MEMORY ALLOCATION

Think of it as renting a warehouse:

- ❑ You decide how much space you need at runtime.
- ❑ You must return it (free it) when done.
- ❑ Useful for handling large or unknown data sizes.

MALLOC() – ALLOCATE MEMORY

malloc()

- The name malloc stands for "memory allocation".
- The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.
- **Syntax of malloc()**

  ptr=(cast-type*)malloc(byte-size)

MALLOC() – ALLOCATE MEMORY

Like booking a warehouse without cleaning it.

- Allocates memory but does not initialize it.

MALLOC() – ALLOCATE MEMORY

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
        int *ptr = (int *)malloc(5 * sizeof(int));  // Allocates space for 5 integers
        if (ptr == NULL)
                {
                printf("Memory allocation failed");
                return 1;
                }
        ptr[0] = 10;  // Assigning value
        printf("%d", ptr[0]);

        free(ptr);  // Free allocated memory
        return 0;
}
```

CALLOC() – ALLOCATE & INITIALIZE MEMORY

calloc()

- The name calloc stands for "contiguous allocation".
- The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size and sets all bytes to zero.
- Syntax of calloc()

```
ptr=(cast-type*)calloc(n,element-size);
```

CALLOC() – ALLOCATE & INITIALIZE MEMORY

Like booking a warehouse and getting it cleaned before use.

- Allocates memory and initializes it to zero.

EXAMPLE:

```
int *ptr = (int *)calloc(5, sizeof(int));  // Allocates space for 5 integers, all set to 0
```

REALLOC() – RESIZE MEMORY

realloc()

- If the previously allocated memory is insufficient or more than sufficient.
- Then, you can change memory size previously allocated using realloc().

Syntax of realloc()

```
ptr=realloc(ptr,newsize);
```

REALLOC() – RESIZE MEMORY

Like upgrading your warehouse if you need more space.

- Expands or shrinks previously allocated memory.

```
int *ptr = (int *)malloc(2 * sizeof(int));
ptr = (int *)realloc(ptr, 4 * sizeof(int));  // Expanding to 4 integers
```

FREE() – RELEASE MEMORY

free()

- Dynamically allocated memory with either calloc() or malloc() does not get return on its own.
- The programmer must use free() explicitly to release space.

syntax of free()

```
free(ptr);
```

- This statement cause the space in memory pointer by ptr to be deallocated.

FREE() – RELEASE MEMORY

Like giving back your rented warehouse after use.

- Always free memory after use to prevent memory leaks.

```
free(ptr);  // Releases allocated memory
```

DIFFERENCE BETWEEN MALLOC AND CALLOC

	Malloc	Calloc
Function	allocates "size" bytes of memory.	allocates a region of memory large enough to hold "n elements" of "size" bytes each.
Syntax	void *malloc (size_in_bytes);	void *calloc (number_of_blocks, size_of_each_block_in_bytes);
Contents of allocated memory	The contents of allocated memory are not changed. i.e., the memory contains unpredictable or garbage values. This presents a risk.	The allocated region is initialized to zero.

DIFFERENCE BETWEEN MALLOC AND CALLOC

Number of Arguments	1	2
Allocation	malloc() doesn’t initialize the allocated memory.	calloc() allocates the memory and also initializes the allocates memory to zero.

DRAWBACK OF POINTERS

If used incorrectly, pointers can lead to bugs that are difficult to unearth. For e.g., if you use a pointer to read a memory location but that pointer is pointing to an incorrect location then you may end up reading a wrong value.

DRAWBACK OF POINTERS

Wild pointer :

- Think of a wild pointer as a loose cannon.
- It is not initialized but still points to some random memory.
- Using it causes unpredictable behavior or crashes.
- **FIX:** Always initialize a pointer before using it:

Memory leak :

- Imagine renting a warehouse but forgetting to return it.
- Memory is allocated but never freed, causing the program to consume more and more memory.
- Fix: Always free memory after use:

DRAWBACK OF POINTERS

Dangling pointer :

- Think of it as using an old phone number that no longer exists.
- A pointer still points to freed memory.
- Accessing it causes unpredictable issues.
- **Fix:** After freeing, set the pointer to NULL:

Memory corruption :

- Like writing outside the lines in a notebook, ruining other pages.
- Happens when you write outside allocated memory.
- **Fix:** Always ensure you access within allocated bounds.

DRAWBACK OF POINTERS

🔷 Summary Table

Issue	Cause	Fix
Wild Pointer	Uninitialized pointer	Initialize pointer (`NULL` or allocate memory)
Memory Leak	Forgetting to free memory	Always call `free(ptr)`
Dangling Pointer	Using freed memory	Set `ptr = NULL` after `free(ptr)`
Memory Corruption	Writing out of bounds	Stay within allocated limits

THANK YOU

MOTIVATION GETS YOU STARTED, DISCIPLINE KEEPS YOU GOING.